UNIVERSITÀ DI PARMA

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Corso di Laurea Triennale in Informatica

Titolo in Italiano

Candidato:
Saverio Mattia Merenda

Relatore:
Prof. Vincenzo Arceri

Anno Accademico 2023–2024

A me medesimo.

Indice

Introduzione **1**

1 Background **3**
1.1 Blockchain 3
1.1.1 Funzionamento 4
1.1.2 Consenso 4
1.1.3 Classificazione delle blockchain 6
1.1.4 Chiave pubblica e chiave privata 6
1.2 Ethereum 7
1.2.1 Etherem vs Bitcoin 7
1.2.2 Funzionamento 8
1.3 Smart Contracts 11
1.3.1 Esempio di smart contract 12
1.4 Ethereum Virtual Machine 14
1.5 EVM Bytecode 15
1.6 Alterazione del flusso di esecuzione 16
1.6.1 Esempio di alterazione 16
1.6.2 Pushed jumps vs Orphan jumps 16
1.7 Vulnerabilità di rientranza 17
1.7.1 Attacco a The DAO 17

Conclusione **19**

Bibliografia **21**

Elenco delle figure

1.1 Funzionamento della blockchain. 4
1.2 Il processo IFTTT negli smart contracts TODO sistemare con photoshop questa figura 12

Elenco degli algoritmi

1 Esempio di smart contract in Solidity. 13

Introduzione

Ethereum [Wood et al., 2014] è una piattaforma blockchain che consente la creazione di un ambiente globale decentralizzato, in cui gli utenti possono sviluppare strumenti digitali sicuri utilizzando la valuta nativa Ether (ETH) per transazioni e servizi computazionali.

La rete Ethereum è costituita da nodi interconnessi che seguono regole definite nel protocollo Ethereum, supportando una vasta gamma di comunità, applicazioni, organizzazioni e attività digitali accessibili a chiunque abbia una connessione internet.

La caratteristica principale di Ethereum è la capacità di eseguire smart contract tramite la Ethereum Virtual Machine (EVM), consentendo funzionalità avanzate al di là delle transazioni finanziarie di base (e.g. Bitcoin [Nakamoto, 2008]). Gli smart contract sono programmi immutabili memorizzati sulla blockchain che eseguono azioni predefinite senza intervento umano.

Tuttavia, garantire la sicurezza e l'affidabilità degli smart contract è essenziale, poiché non possono essere modificati una volta deployati. La presenza di bug o vulnerabilità potrebbe causare gravi conseguenze, come perdite di fondi o azioni indesiderate.

Per garantire la qualità degli smart contract, vengono sviluppate tecniche di analisi e verifica del codice. L'analisi statica, ad esempio, identifica potenziali problemi di sicurezza senza eseguire effettivamente il codice. Tuttavia, nel contesto di Ethereum, la costruzione di un Control Flow Graph (CFG) affidabile è complessa a causa delle destinazioni dinamiche dei salti (Jump Orfane) durante l'esecuzione del EVM bytecode.

In questo elaborato viene presentato EVMLiSA [1], un software in grado di costruire un CFG sound e affidabile (TODO si spera) di smart contract eseguibili su EVM.

[1] `https://github.com/lisa-analyzer/evm-lisa`.

Capitolo 1

Background

TODO scrivere riassuntino

1.1 Blockchain

Il concetto di blockchain è stato introdotto per la prima volta nel 2008 da Satoshi Nakamoto con il whitepaper di Bitcoin [Nakamoto, 2008] e successivamente implementato nel 2009 come tecnologia alla base di Bitcoin.

È importante sottolineare che Bitcoin e blockchain sono due entità separate: Bitcoin rappresenta solo una delle molteplici applicazioni costruite sulla tecnologia blockchain.

La blockchain è un registro digitale, decentralizzato e distribuito su una rete, strutturato come una catena di blocchi responsabili dell'archiviazione dei dati. Questi dati possono comprendere informazioni di valore economico o intere applicazioni digitali. È possibile aggiungere nuovi blocchi di informazioni, ma non è possibile modificare o rimuovere i blocchi precedentemente aggiunti alla catena.

In questo ambiente, la crittografia e i protocolli di consenso garantiscono sicurezza e immutabilità. Il risultato è un sistema aperto, neutrale, affidabile e sicuro, dove la nostra capacità di utilizzare e fidarci del sistema non dipende dalle intenzioni di nessun individuo o istituzione.

La blockchain può essere considerata parte della famiglia delle Distributed Ledger Technology (DLT): tutti i partecipanti alla rete hanno accesso al registro distribuito e al suo registro immutabile delle transazioni. Questo registro condiviso consente di registrare le transazioni una sola volta, eliminando la duplicazione degli sforzi tipica delle reti aziendali tradizionali [Chiap, 2019].

1.1.1 Funzionamento

La blockchain è composta da una serie di blocchi, aggiunti uno dopo l'altro in modo sequenziale. Ogni blocco contiene una prova matematica, resa possibile dall'utilizzo della crittografia, che ne assicura la sequenzialità tramite un hash del blocco precedente.

Sebbene il concetto di catena di blocchi sia comune a quasi tutti i sistemi blockchain, la struttura e il contenuto dei blocchi possono variare a seconda dello scopo della blockchain stessa. Ad esempio, i blocchi della blockchain di Bitcoin possono essere diversi da quelli della blockchain di Ethereum.

I blocchi sono interconnessi tramite funzioni di hash crittografiche (e.g. SHA-256 [TODO forse andrebbe spiegato?]), creando un legame matematico tra di loro. Questo processo fornisce un modo conveniente per esprimere l'intero contenuto della blockchain in una singola stringa di lunghezza definita.

Ogni blocco contiene informazioni specifiche (e.g. le transazioni) e l'hash del blocco precedente (Figura 1.1). Modificare qualsiasi informazione in un blocco altererebbe l'hash del blocco e, di conseguenza, tutti gli hash successivi, rendendo immediatamente evidente l'alterazione.

Ogni nuovo blocco rafforza la verifica dei blocchi precedenti, rendendo la blockchain a prova di manomissione e fornendo un registro affidabile delle transazioni su cui gli utenti possono fare affidamento [Chiap, 2019].

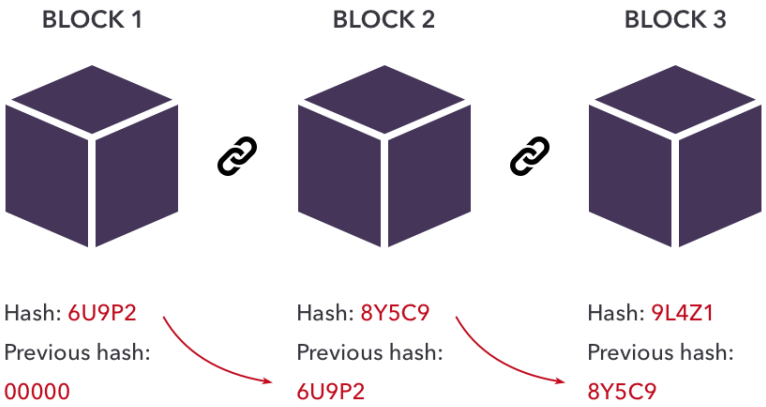

Figura 1.1: Funzionamento della blockchain.

1.1.2 Consenso

Il consenso è il processo mediante il quale i nodi della rete raggiungono un accordo su ciò che è avvenuto all'interno della blockchain. Questo accordo

rappresenta l'unica verità sullo stato attuale della blockchain e viene ottenuto attraverso un accordo generale tra i partecipanti della rete. Il consenso non è un evento istantaneo, ma un processo continuo che coinvolge vari partecipanti con ruoli e responsabilità specifici. Si basa su principi matematici ed e conomici per incentivare tutti i membri a raggiungere un accordo su un unico dato [Chiap, 2019].

Nella pratica, il consenso richiede che i nuovi blocchi siano approvati dalla maggioranza dei nodi della rete, che sono distribuiti su una vasta rete di computer. Questo processo può avvenire attraverso due meccanismi principali: la Proof of Work (PoW) e la Proof of Stake (PoS).

- Proof of Work (PoW): È il sistema di consenso utilizzato principalmente da Bitcoin. Qui, i nodi della rete, noti come "miner", competono per risolvere complessi problemi crittografici. Il primo nodo che risolve con successo il problema crittografico viene premiato con una quantità specifica di criptovaluta. La difficoltà di generare l'hash viene modificata man mano che la rete si espande, in modo che i nuovi blocchi vengano creati e approvati a un ritmo costante al variare della potenza di calcolo della rete. La difficoltà di generare blocchi nella blockchain di Bitcoin, ad esempio, viene regolata modificando il numero di zeri con cui devono iniziare, garantendo che un nuovo hash venga trovato solo una volta ogni dieci minuti circa dall'intera rete. Questi problemi sono progettati per richiedere una grande quantità di potenza di calcolo e l'energia utilizzata in questo processo è sia una caratteristica distintiva che un punto di vulnerabilità di PoW. Se da un lato rende la blockchain sicura, poiché sarebbe estremamente costoso e poco pratico per un singolo attaccante possedere la maggior parte della potenza di calcolo della rete, dall'altro richiede una notevole quantità di energia elettrica [Tarpini, 2023].

- Proof of Stake (PoS): È un meccanismo di consenso in cui i nodi della rete, chiamati "validatori", competono per essere selezionati per convalidare il prossimo blocco. La probabilità di essere selezionati è proporzionale alla quantità di criptovaluta che il nodo ha depositato nella blockchain. Quando un validatore viene selezionato, "mette in gioco" (stake) i propri fondi (token). Se il validatore non adempie ai propri doveri o tenta di commettere una frode, rischia di perdere i suoi fondi. PoS non richiede una quantità significativa di energia elettrica come PoW, ma presenta un problema di monopolio della ricchezza, poiché coloro che possiedono più token hanno maggiori probabilità di essere selezionati per convalidare il blocco successivo e ricevere la ricompensa. Questo problema è mitigato con l'introduzione di Delegated Proof

of Stake (DPoS), che prevede un sistema di voto per la selezione dei validatori, rendendo il processo più democratico [Tarpini, 2023].

1.1.3 Classificazione delle blockchain

Le blockchain possono essere categorizzate in vari tipi in base alla loro accessibilità e ai requisiti di utilizzo:

- Permissionless: Questo tipo di blockchain permette a chiunque di accedervi senza restrizioni, senza la necessità di identificarsi. Esempi notevoli sono Bitcoin ed Ethereum. Tuttavia, questa libertà può portare a problemi di dimensionamento e sicurezza, poiché ogni nodo della rete deve elaborare e convalidare l'intera blockchain, rendendo anche più probabile l'ingresso di attori malevoli [Permisison.io, 2021].

- Permissioned: Al contrario, nelle blockchain ad accesso limitato, l'accesso è controllato e richiede l'autenticazione da parte di un'autorità centrale. Questo approccio riduce i problemi di dimensionamento e sicurezza riscontrati nelle blockchain aperte. Tuttavia, l'intervento di un'autorità centrale compromette parzialmente il principio di decentralizzazione [Permisison.io, 2021].

Ulteriormente, le blockchain possono essere suddivise in base alle loro applicazioni e requisiti specifici:

- Public: Le blockchain pubbliche incoraggiano la partecipazione di tutti gli utenti, offrendo incentivi come ricompense in criptovaluta. Sono trasparenti e flessibili. Esempi ben noti includono Bitcoin ed Ethereum [Sharma, 2023].

- Private: Le blockchain private sono utilizzate all'interno di reti aziendali o organizzative, spesso per migliorare l'efficienza dei processi interni. Sono meno trasparenti rispetto alle blockchain pubbliche e richiedono solitamente un'autorizzazione per accedervi [Sharma, 2023].

- Consortium: Queste blockchain sono progettate per la collaborazione tra più entità, come aziende o istituzioni. La gestione dell'accesso è condivisa tra i partecipanti, garantendo un certo livello di decentralizzazione senza dipendenza da un'autorità centrale [Sharma, 2023].

1.1.4 Chiave pubblica e chiave privata

TODO non so se metterlo o no

1.2 Ethereum

Ethereum è una blockchain aperta (permissionless), accessibile a chiunque (pubblica) e con il codice sorgente disponibile liberamente (open-source). È stata ideata per la prima volta da Vitalik Buterin nel 2013 [Buterin, 2013], con l'obiettivo di creare una blockchain in grado di eseguire una vasta gamma di programmi generici.

Dal punto di vista tecnico, Ethereum può essere considerato una sorta di enorme macchina virtuale globale e "infinita", che opera seguendo uno stato singolo accessibile da qualsiasi parte del mondo e una macchina virtuale che applica le modifiche a tale stato.

Tuttavia, in termini più pratici, Ethereum si presenta come un'infrastruttura informatica decentralizzata, aperta a tutti e basata su codice sorgente accessibile [1], che consente l'esecuzione di programmi denominati Smart Contracts (Sezione 1.3). Utilizza una blockchain per tenere traccia e registrare le variazioni di stato del sistema, utilizzando la criptovaluta nativa (chiamata Ether) per misurare e regolare i costi delle risorse di elaborazione.

Attraverso la piattaforma Ethereum, gli sviluppatori hanno la possibilità di creare applicazioni decentralizzate con funzioni economiche integrate, offrendo non solo elevata disponibilità, verificabilità, trasparenza e neutralità, ma anche la riduzione o l'eliminazione della censura e alcuni rischi associati alle controparti tradizionali [Antonopoulos and Wood, 2018].

1.2.1 Etherem vs Bitcoin

Ethereum presenta numerose caratteristiche comuni con altre blockchain aperte: una rete peer-to-peer che collega gli utenti, un algoritmo di consenso per mantenere gli aggiornamenti di stato sincronizzati (inizialmente basato su un consenso proof-of-work, ma con l'avvento dell'aggiornamento "The Merge" [2], il consenso è passato a proof-of-stake), l'uso di tecniche crittografiche come firme digitali e hash, e una valuta digitale (ETH).

Tuttavia, sia lo scopo che la struttura di Ethereum si differenziano notevolmente da quelle delle blockchain precedenti, come Bitcoin.

Il principale obiettivo di Ethereum non è solo quello di fungere da sistema di pagamento digitale. Anche se l'ether è fondamentale per il funzionamento di Ethereum, essa viene considerata una "valuta di utilità" utilizzata per pagare l'utilizzo della piattaforma Ethereum come un computer globale.

[1] `https://github.com/ethereum/go-ethereum`
[2] `https://ethereum.org/it/roadmap/merge/`

A differenza di Bitcoin, che dispone di un linguaggio di scripting limitato, Ethereum è stato progettato per essere una blockchain programmabile per scopi generici, con una macchina virtuale EVM in grado di eseguire codice di complessità arbitraria e illimitata. Mentre il linguaggio di scripting di Bitcoin si limita principalmente a valutazioni semplici (true/false) delle condizioni di spesa di un utente, il linguaggio di Ethereum (Solidity [3]) è quasi completo in termini di capacità computazionale (Turing completeness), consentendo ad Ethereum di funzionare come un computer per scopi generali [Antonopoulos and Wood, 2018].

1.2.2 Funzionamento

Sulla blockchain Ethereum sistono due tipi principali di account utilizzabili per gestire le transazioni e l'interazione sulla rete: gli Externally Owned Account e i Contract Account.

Gli Externally Owned Account (EOA) sono controllati direttamente dagli utenti e sono associati a una coppia di chiavi crittografiche, pubblica e privata. Questi account consentono agli individui di ricevere e inviare ether e di partecipare alle transazioni sulla rete. Le transazioni tra due account esterni coinvolgono esclusivamente lo scambio di ether e non comportano alcun costo di creazione dell'account [Nico et al., 2023a].

I Contract Account, invece, sono associati agli smart contract. Questi account contengono il codice dello smart contract e vengono attivati quando ricevono una transazione. Le transazioni con questi account possono coinvolgere l'esecuzione di codice dello smart contract, oltre allo scambio di ether e comportano un costo di creazione in quanto richiedono risorse di calcolo e di archiviazione sulla rete Ethereum [Nico et al., 2023a].

Affinché una transazione venga effettuata su Ethereum, il mittente deve conoscere l'indirizzo del destinatario, detto anche chiave pubblica dell'account, e firmare digitalmente la transazione con la propria chiave privata. Questo processo dimostra che il richiedente della transazione è il legittimo proprietario dell'account.

Le transazioni sono essenzialmente istruzioni crittograficamente firmate da un account che iniziano una modifica dello stato della rete Ethereum. Inoltre, per "transazione" si intende una transazione approvata e inclusa in un blocco della blockchain.

Poiché le transazioni sono atomiche, esse devono essere eseguite completamente prima di apportare cambiamenti allo stato globale della rete. Questo significa che tutte le istruzioni all'interno della transazione devono essere

[3] `https://docs.soliditylang.org/en/latest/`

valide. Se una qualsiasi istruzione fallisce, gli effetti della transazione vengono annullati e lo stato viene ripristinato al momento precedente (rollback), come se la transazione non fosse mai avvenuta. Anche se una transazione fallisce, viene comunque registrata come tentata, ma non influenza lo stato complessivo della rete [Nico et al., 2023b].

Ogni transazione su Ethereum comporta un costo proporzionale alla sua complessità computazionale, misurato in "gas". Possiamo pensare al gas come al carburante necessario per far funzionare le operazioni sulla blockchain, simile al carburante utilizzato da un'auto per percorrere una determinata distanza.

Ogni operazione sulla blockchain ha un costo specifico in gas. Ad esempio, calcolare un hash o fare una somma di due numeri richiede una differente quantità di gas (30 e 3, rispettivamente).

Il gas è misurato in "wei" (1 wei $= 1^{-18}$ ETH) ed è strettamente legato alle transazioni. Ogni transazione su Ethereum ha due parametri: il prezzo del gas (gas price) e il limite del gas (gas limit), che rappresentano rispettivamente il prezzo che si è disposti a pagare per unità di gas e la quantità massima di gas utilizzabile.

A differenza di Bitcoin, dove esiste un limite alla dimensione massima di un blocco, su Ethereum si fa riferimento al limite di gas, che determina la massima quantità di calcoli che la Ethereum Virtual Machine deve eseguire per blocco.

Immaginiamo di dover eseguire una transazione che richiede 10 gas e abbiamo deciso di pagare 100 wei per ogni gas: il costo totale della transazione sarà di 1000 wei (10×100). Se aumentiamo il prezzo del gas a 1000 wei per gas, il costo totale della transazione sarà di 10000 wei (10×1000). I validatori della rete Ethereum sono liberi di scegliere le transazioni da includere nel nuovo blocco, e generalmente cercano di massimizzare il profitto. Di conseguenza, le transazioni con un prezzo del gas più elevato hanno una priorità maggiore di essere aggiunte al blocco successivo.

Se una transazione richiede 15 gas ma abbiamo impostato un limite di 10 gas, l'esecuzione si interromperà quando verrà raggiunto il limite, e tutto il gas andrà perso. Se, invece, il limite di gas è superiore a quello utilizzato dalla transazione, il gas in eccesso verrà restituito al mittente.

Quindi, il costo totale di una transazione può essere calcolato con la seguente formula:

$$\text{Costo transazione} = \text{Limite di gas} \times \text{Prezzo del gas}$$

Oltre a compensare i validatori, il costo delle transazioni su Ethereum serve anche a proteggere la blockchain da attacchi, come il DDoS, e da errori di programmazione negli smart contract che potrebbero sovraccaricare

il sistema. Ad esempio, se una computazione finisse in un ciclo infinito, la gestione del gas interromperebbe l'esecuzione una volta che il gas disponibile è esaurito. Questa caratteristica ci consente di definire Ethereum come un linguaggio (quasi) Turing-completo, poiché siamo sicuri che ogni programma in esecuzione terminerà [Chiap, 2019].

1.3 Smart Contracts

Il concetto di smart contract è stato introdotto per la prima volta da Nick Szabo nel 1996, definendolo come "un protocollo di transazione digitale che esegue i termini di un contratto" [Szabo, 1996]. Lo scopo principale di uno smart contract è quello di automatizzare l'adempimento delle condizioni contrattuali, riducendo al minimo il rischio di azioni malevole e la necessità di fidarsi degli intermediari (rischio di controparte).

Un contratto, in generale, è un accordo legalmente vincolante tra due o più parti e svolge un ruolo fondamentale nell'instaurare fiducia tra i soggetti coinvolti in una transazione [4]. Può essere tanto semplice quanto un biglietto dell'autobus o molto complesso come un contratto di lavoro.

Nel contesto delle blockchain, uno smart contract è un programma che replica tutte le caratteristiche di un contratto tradizionale, ma viene memorizzato ed eseguito all'interno di una blockchain. Si tratta di un agente autonomo che risiede su una blockchain, senza la necessità di un'entità esterna per valutare le condizioni e prendere decisioni. Questo ruolo è sostituito dal consenso della rete. Gli smart contract stabiliscono le regole e le fanno rispettare automaticamente alle parti coinvolte, senza la necessità di autorità centrali. Quando le condizioni del contratto vengono soddisfatte, lo smart contract esegue autonomamente azioni specifiche, come ad esempio il trasferimento di denaro [Chiap, 2019].

Un smart contract può essere paragonato a un'applicazione IFTTT (If This Then That) che risponde a eventi specifici (Figura 1.2).

Gli smart contract, solitamente redatti in un linguaggio di alto livello come Solidity, necessitano di essere compilati nel bytecode di basso livello eseguito nell'EVM per poter essere eseguiti. Dopo la compilazione, vengono distribuiti sulla piattaforma Ethereum attraverso una speciale transazione di creazione del contratto, inviata all'indirizzo di creazione del contratto `0x0`, detto *zero address*. Ogni contratto è identificato da un indirizzo Ethereum, derivato dalla transazione di creazione del contratto in funzione dell'account di origine e del nonce. Come già menzionato nella sezione 1.2.2, a differenza degli EOAs, gli account creati per nuovi smart contract non hanno chiavi associate.

Gli smart contract vengono eseguiti solo quando vengono richiamati da una transazione. Ogni smart contract in Ethereum viene eseguito a seguito di una transazione avviata da un EOA. Un contratto può richiamare un altro contratto, che a sua volta può richiamare un altro contratto e così via, ma il

[4] `https://www.treccani.it/enciclopedia/contratto/`

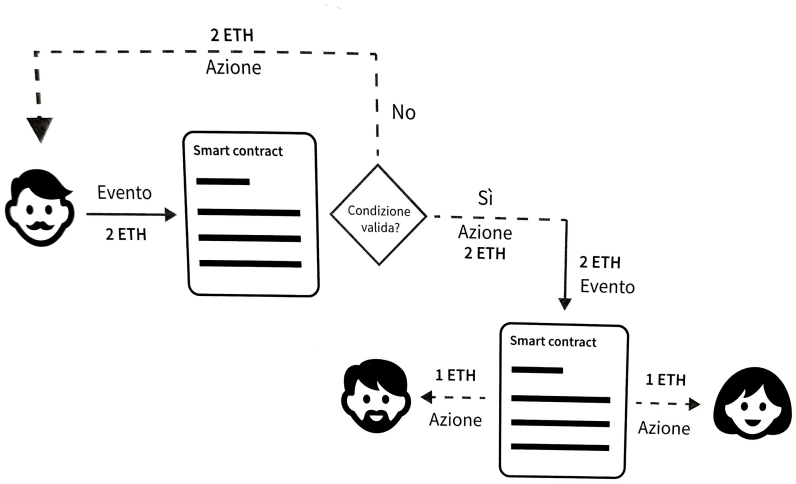

Figura 1.2: Il processo IFTTT negli smart contracts
TODO sistemare con photoshop questa figura

primo contratto in questa catena di esecuzione sarà sempre stato richiamato da una transazione proveniente da un EOA.

Le transazioni sono atomiche, indipendentemente dal numero di smart contract richiamati o dalle azioni che eseguono. Ogni transazione viene eseguita nella sua totalità e le eventuali modifiche allo stato globale vengono registrate solo se l'esecuzione termina con successo. Se l'esecuzione fallisce a causa di un errore, tutti gli effetti vengono annullati e la transazione viene registrata come tentata.

È importante notare che una volta distribuito, il codice di un contratto non può essere modificato. Approfondiremo ulteriormente questo aspetto in seguito.

1.3.1 Esempio di smart contract

Ma adesso vediamo un esempio di smart contract [5] nell'Algoritmo 1.

Questo smart contract è composto da due contratti: *ReceiveEther* e *SendEther*.

Il contratto *ReceiveEther* è progettato per ricevere *ether*. Contiene due funzioni: `receive` e `fallback`. La funzione `receive` viene chiamata quando viene inviato ether al contratto e non ci sono dati associati alla transazione. Questo è possibile tramite l'invio di ether direttamente all'indirizzo del contratto senza specificare alcuna funzione. La funzione `fallback` viene invece

[5]Tratto da: `https://solidity-by-example.org/sending-ether/`

chiamata quando viene inviato ether al contratto e sono presenti dati associati alla transazione (e.g. TODO fare esempio). Entrambe le funzioni accettano ether e sono contrassegnate come `payable` per indicare che possono ricevere fondi. È importante specificare che un contratto che riceve ether deve avere almeno una di queste due funzioni. La funzione `getBalance` restituisce il saldo attuale del contratto in ether.

Il contratto *SendEther* è progettato invece per inviare ether. Contiene una sola funzione `sendViaCall` che invia ether a un altro indirizzo Ethereum. Questa funzione prende in input l'indirizzo del destinatario e l'ammontare di ether da inviare. Utilizza la funzione `call` per inviare ether al destinatario e restituisce un booleano che indica se l'operazione è stata eseguita con successo. Infine, la funzione `require` permette di verificare che l'invio sia andato a buon fine, in caso contrario l'esecuzione viene interrotta con un messaggio di errore.

Algoritmo 1 Esempio di smart contract in Solidity.

```
pragma solidity ^0.8.24;

contract ReceiveEther {
    receive() external payable {}

    fallback() external payable {}

    function getBalance() public view returns (uint256) {
        return address(this).balance;
    }
}

contract SendEther {
    function sendViaCall(address payable _to) public payable {
        (bool sent, bytes memory data) = _to.call{value: msg.value}("");
        require(sent, "Failed to send Ether");
    }
}

```

1.4 Ethereum Virtual Machine

1.5 EVM Bytecode

1.6 Alterazione del flusso di esecuzione

1.6.1 Esempio di alterazione

1.6.2 Pushed jumps vs Orphan jumps

1.7 Vulnerabilità di rientranza

1.7.1 Attacco a The DAO

Conclusione

Conclusione che riassume il lavoro svolto ed eventuali lavori futuri.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliografia

[Antonopoulos and Wood, 2018] Antonopoulos, A. M. and Wood, G. (2018). *Mastering ethereum: building smart contracts and dapps*. O'reilly Media.

[Buterin, 2013] Buterin, V. (2013). Ethereum white paper. *GitHub repository*, 1:22–23.

[Chiap, 2019] Chiap, G. (2019). Blockchain: tecnologia e applicazioni per il business.

[Nakamoto, 2008] Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system.

[Nico et al., 2023a] Nico et al. (2023a). Ethereum accounts.

[Nico et al., 2023b] Nico et al. (2023b). Ethereum smart contracts.

[Permisison.io, 2021] Permisison.io (2021). Permissioned vs. permissionless blockchains explained.

[Sharma, 2023] Sharma, T. K. (2023). Types of blockchains explained - public vs. private vs. consortium.

[Szabo, 1996] Szabo, N. (1996). Smart contracts: building blocks for digital markets. *EXTROPY: The Journal of Transhumanist Thought,(16)*, 18(2):28.

[Tarpini, 2023] Tarpini, D. (2023). Risoluzione di jump orfane in bytecode evm tramite stack simbolici e interpretazione astratta.

[Wood et al., 2014] Wood, G. et al. (2014). Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus

a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.